
Sungnam-Si #109 Bus ArrivalTime Prediction



컴퓨터공학부
20213803 3학년 허준호

목 차

- 요약
- 서론
- 배경 - 마을버스의 불규칙성
- 관련연구 - 도로 데이터 분석
- 제안하는 아이디어 - DNN 기반 회귀 모델 설계
- 실험 및 결과
- 결론
- 참조문헌

요약

본 연구는 불규칙한 배차 간격과 교통 상황으로 인해 발생하는 마을버스의 도착 예정 시간 오차를 줄이기 위해, 기존의 단순 연산 방식이 아닌 딥러닝(Deep Neural Network) 기반의 예측 모델을 제안합니다. 이를 위해 국토교통부와 기상청의 공공 데이터를 활용하여 교통량과 날씨 정보를 수집하였고, 이를 학습 데이터로 가공하였습니다.

예측 모델의 성능 최적화를 위해 활성화 함수로 ReLU, Sigmoid, 그리고 Swish를 비교 분석하였으며, 과적합 방지를 위해 Batch Normalization과 Dropout 기법을 적용하여 실험을 진행하였습니다.

실험 결과, 평균 약 402초의 오차가 발생하였으며, 이는 간접적으로 수집된 공공 데이터의 정밀도 한계와 데이터의 잡음(Noise)에 기인한 것으로 분석되었습니다.. 본 연구를 통해 저급 데이터 학습의 한계를 확인하고, 향후 고정밀 데이터(GNSS 등)의 필요성과 활성화 함수 선택의 중요성을 시사합니다.

서론

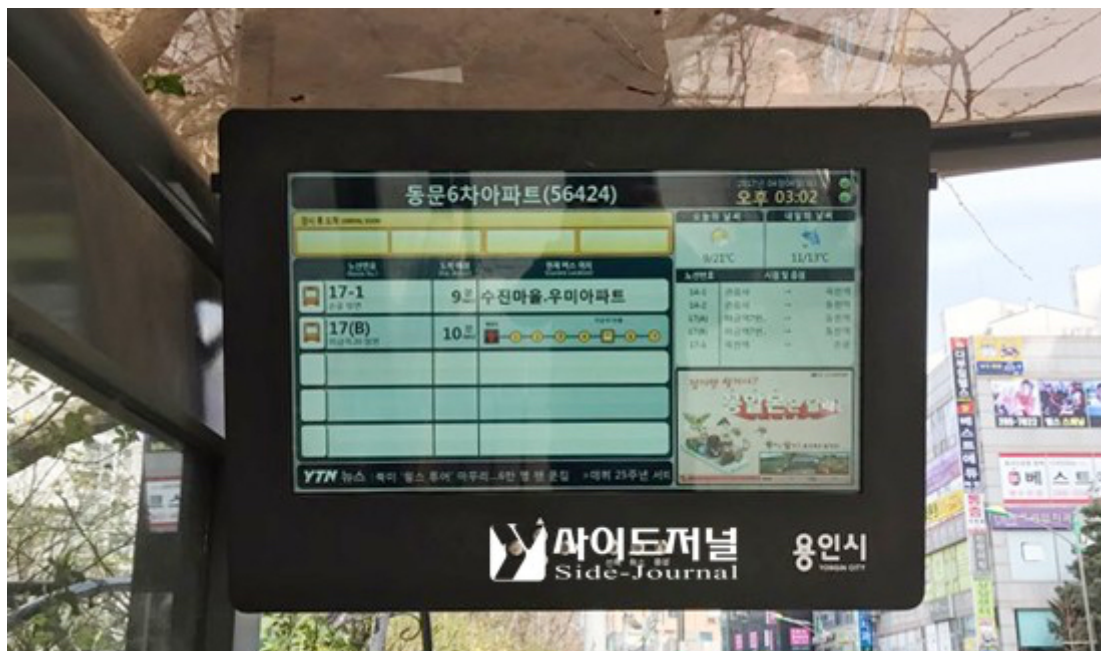
현대 사회에서 버스는 필수적인 대중교통 수단이지만, 정류장의 안내 단말기(BIT)와 모바일 애플리케이션 간의 도착 예정 시간이 상이하여 승객들이 혼란을 겪는 경우가 빈번합니다. 특히 마을버스의 경우 시내버스에 비해 배차 간격이 불규칙하고 교통 변수에 민감하여 정확한 도착 시간을 예측하기 어렵습니다.

본 연구에서는 값비싼 장비 도입 없이, 기존에 접근 가능한 공공 데이터(도로 교통량, 날씨 등)를 활용하여 인공지능이 스스로 교통 패턴을 학습하는 딥러닝(DNN) 기반의 회귀 모델을 제안합니다. 단순한 거리 계산을 넘어, 요일, 시간대, 날씨, 도로 특성 등 다양한 변수 간의 비선형적 관계를 학습함으로써 마을버스의 불규칙성에 대응하고 예측 정확도를 높이는 것을 목표로 합니다.

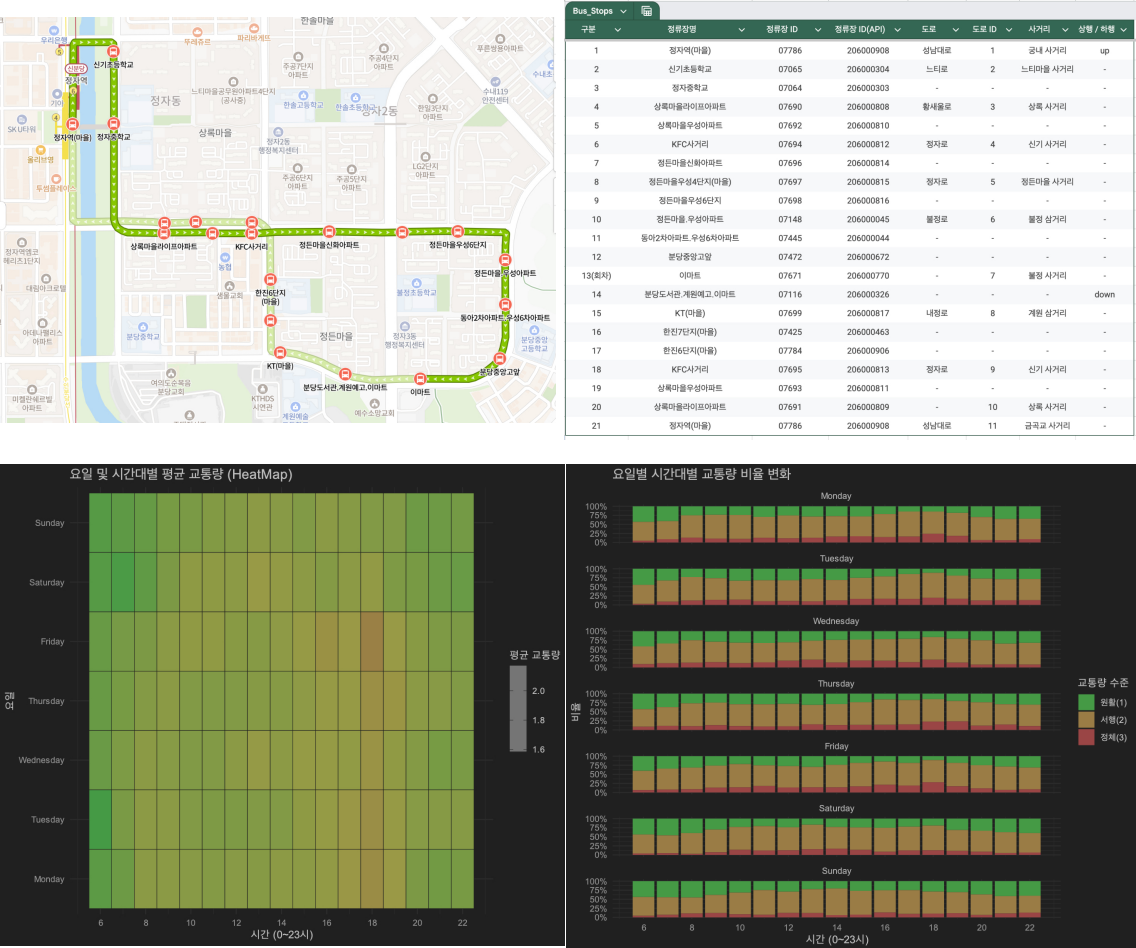
배경 - 마을버스의 불규칙성

연구를 시작하게 된 배경은 버스 정류장에 나와있는 버스 도착시간(안내 단말기)과 모바일 앱에 나와있는 도착시간이 다르다는 것과 어느쪽도 정확하진 않아 한쪽 정보에만 의존하다 버스를 놓치는 경우가 많았다는 것에 기반합니다. 버스 정류장에 있는 전광판의 데이터는 $t = \frac{d}{s}$ 를 이용해 실시간 버스 위치로부터 현재 정류장까지의 남은 시간을 몇분마다 갱신하는 방식이고, 모바일 앱은 앱마다 사용하는 API가 달라 명확하게 공식을 설명할 순 없지만 최근 트렌드는 초정밀버스 위성항법시스템(GNSS)을 버스에 부착해서 제공하는 방식을 사용하고 있습니다. 1초마다 정보가 갱신되어 실시간 버스 위치를 확인할 수 있어 비교적 정확한 정보를 제공받을 수 있습니다.

저는 초정밀버스 서비스를 제공하는 모바일 앱이 아닌 기본적인 API를 제공받아 서비스 되는 앱과 비교해 단순한 예측이 아닌 인공지능 학습을 통한 예측모형을 개발해 마을버스의 불규칙성에 어느정도 대응할 수 있도록 연구를 진행했습니다.



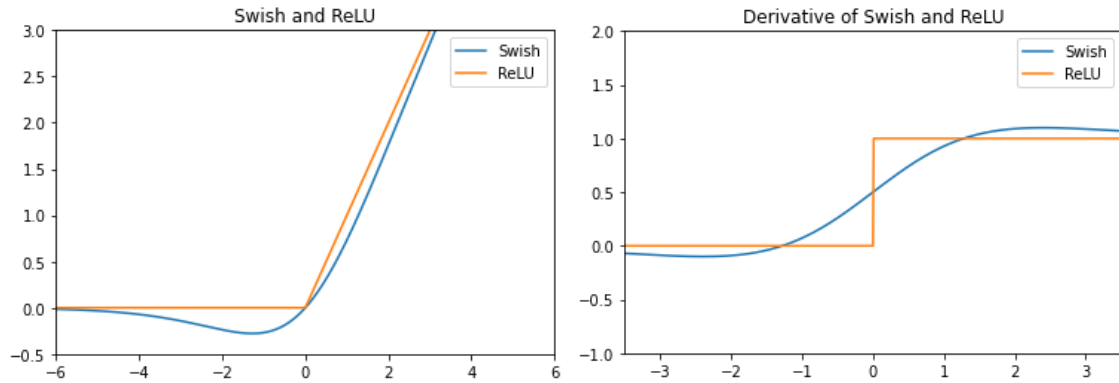
관련연구 - 도로 데이터 분석



버스 도착시간 예측의 신뢰도를 위해 버스 노선의 도로를 자세히 분석해본 결과 국도, 시내도로, 어린이 보호구역 이렇게 3가지 분류군이 나왔고 각각 최저속도와 최고속도가 다른 점을 확인하였습니다. 하지만, 공공데이터포털의 API로는 국도와 고속도로로 한정된 데이터만 제공하고 있어 시내도로에 해당하는 정보는 습득하기가 어려웠습니다. 이를 해결하기 위해 국내 지도 데이터로 가장 많이 사용되는 T-Map 데이터를 사용하려 했으나 이 또한 세부적인 마을 내 도로까진 정보를 제공중이 아니였고 무엇보다 실제 그 도로를 차를 타고 이동해보았을 때 제공받은 데이터와는 완전 다른 결과가 나왔기 때문에 제외하게 되었습니다.

T-Map을 대체하기 위해 더 찾아본 결과 도로교통정보센터의 API를 찾아 1달동안 직접 제작한 프로그램을 개인서버를 이용해 10분 간격으로 데이터를 수집하게 되었습니다. 그 결과가 하단 HeatMap 그래프와 StackedBar 그래프입니다. 분석결과, 평일에는 출근 시간인 아침 8~9시 쯤이 정체되었고 퇴근시간인 오후 5~7시 쯤이 정체됩니다. 주말에는 출퇴근 시간이 없어 정오를 기준으로 평평하게 골고루 분포되어 있는 모습을 보입니다.

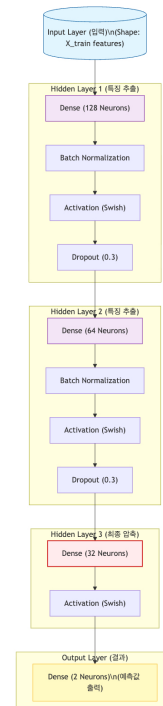
제안하는 아이디어 - DNN 기반 회귀 모델 설계



본 연구에서는 버스 도착 시간을 예측하기 위해 입력 변수(요일, 시간, 날씨, 도로 교통량 등)와 출력 변수 사이의 비선형적인 관계를 학습할 수 있는 Deep Neural Network (DNN) 기반의 회귀(Regression) 모델을 제안합니다. 기존의 선형적인 계산 방식 $t = \frac{d}{s}$ 으로는 해결하기 어려운 도로 상황의 불규칙성을 인공지능이 학습하여 오차를 줄이는 것을 목표로 합니다.

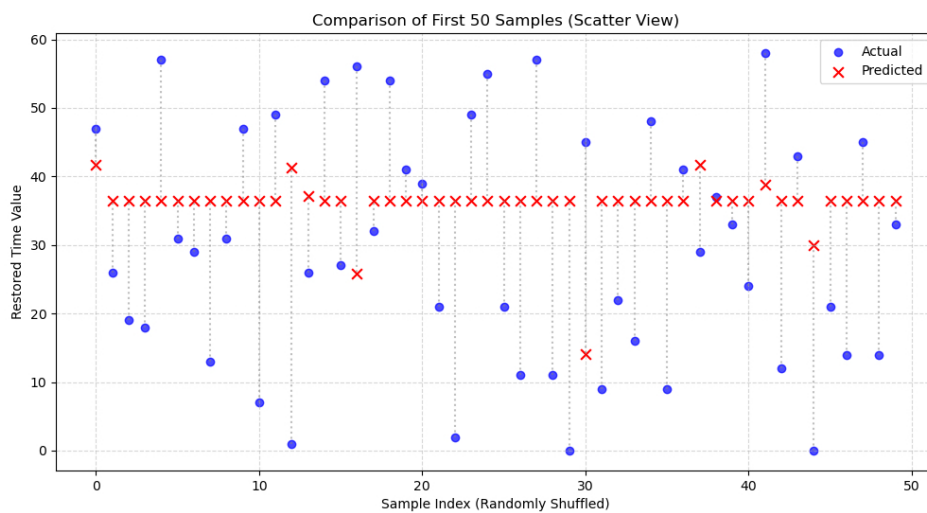
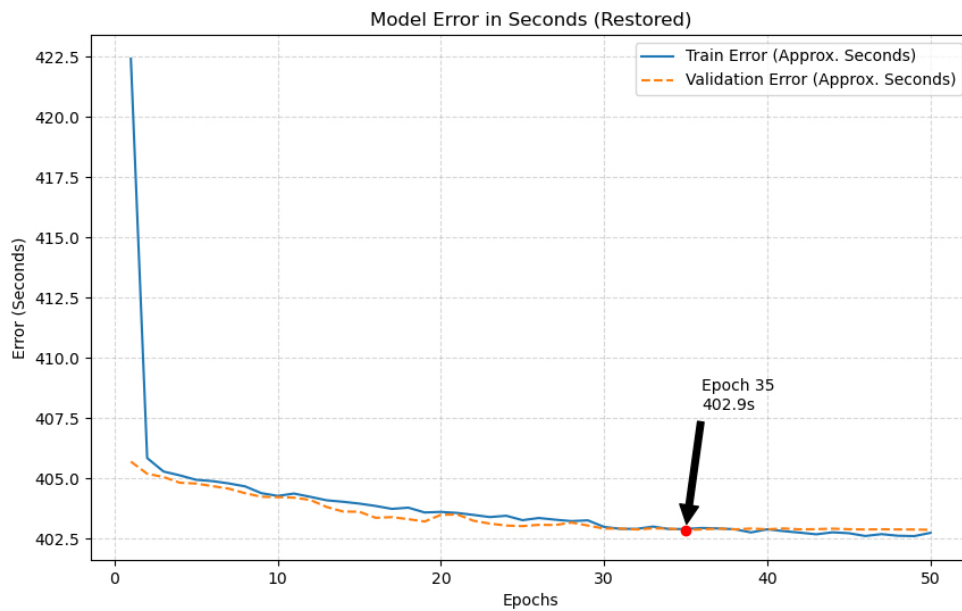
Swish 활성화 함수의 도입기존 딥러닝 모델에서 널리 사용되는 ReLU(Rectified Linear Unit) 함수는 입력값이 음수일 경우 0을 출력하여 뉴런이 비활성화되는 'Dying ReLU' 문제를 야기할 수 있습니다. 본 연구에서는 이를 개선하기 위해 Google Brain 팀이 제안한 Swish 활성화 함수를 도입했습니다.

Swish 함수는 $f(x) = x \cdot \sigma(\beta x)$ 의 형태를 가지며, ReLU와 달리 음수 영역에서도 0이 아닌 값을 출력하여 정보의 손실을 막고 더 유연한 학습을 가능하게 합니다. 본 연구에서는 Swish 함수가 복잡한 도로 데이터의 패턴을 학습하는 데 있어 ReLU나 Sigmoid보다 더 적합한 민감도를 가질 것으로 가정해 적용하였습니다.

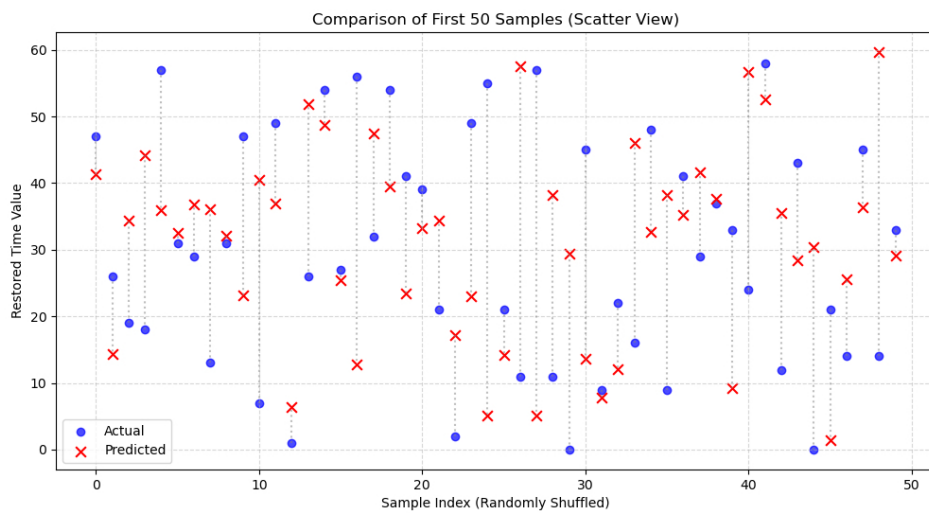
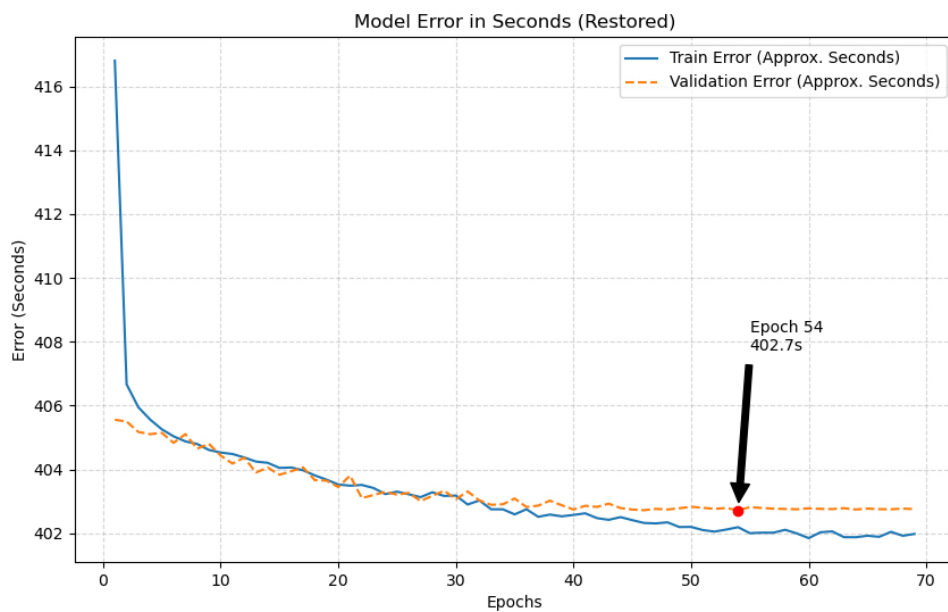


제한된 데이터셋으로 학습할 때 발생할 수 있는 과적합(Overfitting)을 방지하고 모델의 일반화 성능을 높이기 위해, 본 연구에서는 Dropout과 Batch Normalization 기법을 동시에 적용했습니다. 실험을 통해 두 기법을 적절히 혼용했을 때 오차가 가장 최소화됨을 확인하였으며, 이를 최종 모델 구조에 반영하였습니다.

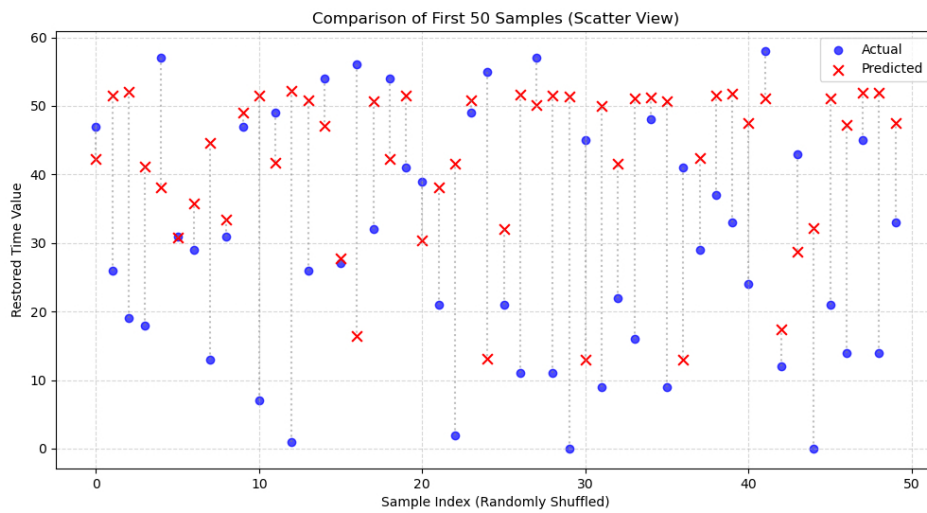
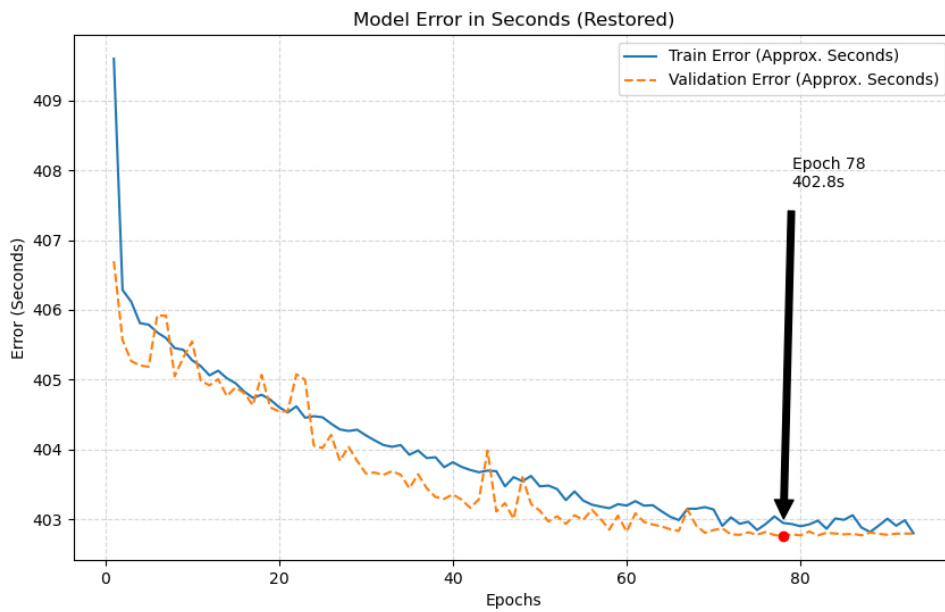
실험 및 결과



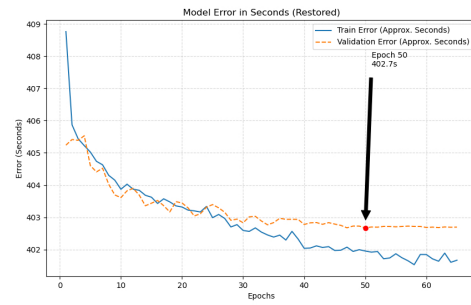
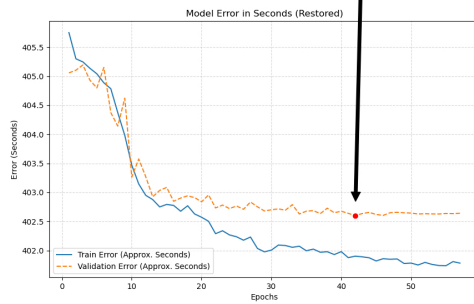
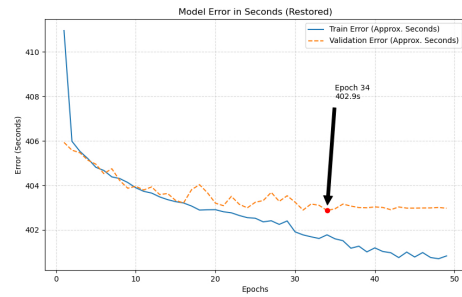
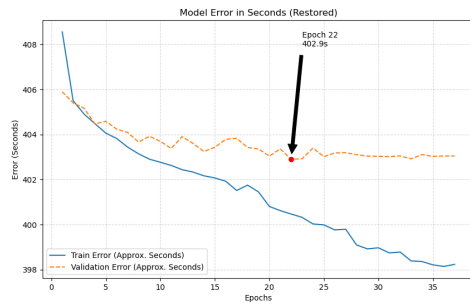
Activation 'ReLU' Accracy 1/50(Dying)



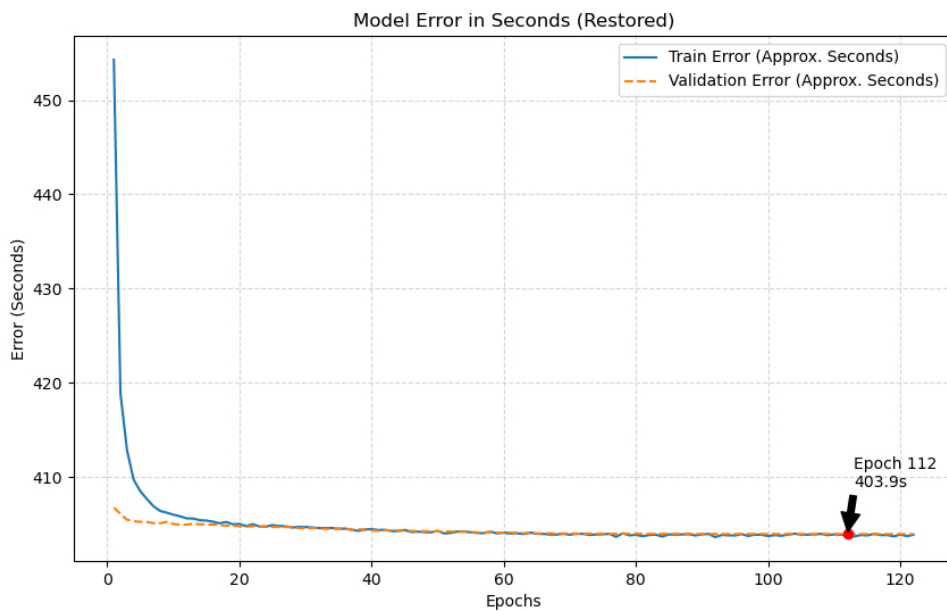
Activation 'Swish' Accracy 5/50



Activation 'Sigmoid' Accracy 3/50



‘Swish’ 함수 기준 Dropout(0.3)을 OFF/ON(1개) 해본 결과(1열), Batch Normalization을 OFF/ON(1개) 해본 결과(2열)



학습을 늘려봤을때의 모습

실험결과, 평균 402초대로 약 6.5분에 해당하는 오차범위가 나타나는 충격적인 모습을 확인하게 되었습니다. 두 번째 랜덤 샘플을 맞추는 테스트를 통해 얼마나 잘 맞추는지도 확인해보았는데, 사실상 공부하지 않은 학생이 시험당일 공부하고 모르는 것은 다 찍어버린 수준의 정확도를 보여주었습니다.

다만, 정확한 데이터가 아니었다는 점과 오류 혹은 더미 데이터가 정말 많아 잡음이 많았고 날씨, 도로, 날짜(Day), 시간(Hour), 주말여부, 출퇴근시간 여부만 가지고 도착시간(Minute)을 예측하라는 어려운 문제를 제시했다는 점을 생각하면 생각보다 상당히 괜찮은 결과라고 판단됩니다.

DNN 은닉층에 주로 선호되는 'ReLU'와 이진분류 모델에 주로 선호되는 'Sigmoid' 그리고 연구중 새로이 발견한 함수인 'Swish'를 비교분석한 결과, Sigmoid와 ReLU의 중간정도의 민감함이 그래프로 아주 잘 표현되고 있다는걸 확인했습니다. 의외로 Sigmoid가 50회 테스트기준 ReLU에 비해 정확도를 보여주었습니다.(이는 데이터 정규화 'MinMaxScaler'로 인해 그런 것으로 판명됨) ReLU의 정확도가 이상한 이유도 추가적으로 연구해본 결과, 학습 초반에 큰 기울기가 들어와 음수 영역으로 빠져버려 생각하는 것을 멈추고 영원히 깨어나지 못하는 'Dying ReLU' 상태가 되어버린 것이라곤 알게 되었습니다.

OverFitting을 막기위해 DropOut을 전부 끈것과 절반만 킨 것, Batch Normalization을 전부 끈것과 절반만 킨 것을 비교해본 결과 둘다 키는 것이 현재 모델에서는 가장 낮은 오차를 보여주었습니다. 하지만 둘다 키는 것은 일반적인 상황에서는 규제가 많아지는 것으로 UnderFitting이 발생한다고 합니다.

마지막으로, 저급데이터를 강제로 많이 학습시킨다면 어떻게되는지 궁금해 추가적으로 연구를 진행했습니다. 그 결과는 마지막 그래프를 보면 완전 평평해지는 모습으로 더 이상 학습이 진행되지 않고 일정한 오차를 보여주는 모습이었습니다. 오차 또한 오히려 소폭 증가한 모습입니다.

결론

데이터 정밀도가 얼마나 중요한지 깨닫게 되었습니다. '초정밀버스'와 같은 직접적으로 데이터를 수집한 방법이 아니었기에 한계는 명확하였고 그 결과를 학습결과를 통해 확인하였습니다. 아이디어 제안사항이었던 'Swish' 함수 또한 이 함수를 적극적으로 활용할만한 정밀한 데이터가 아니었기에 다소 아쉬운 결과가 나타나게 되었습니다.

도로 데이터는 도로구조에 따른 '어느정도의 경향성이 있었다' 정도인 것으로 결론지어집니다. 이유는 경찰청에서 무엇으로 수집한 데이터인지 알 수가 없어 불확실하고 T-Map과 같이 정확도에 있어 불확실성이 여전히 존재했기 때문입니다. 다만, 도로특성이 어느정도 드러난다는 점을 이용해 가중치로 사용해보기로 했으나 큰 효과가 없던 것으로 판명됩니다.

날씨 데이터는 기상청에서 제공된 '측정된 정보'였기에 신뢰도는 높았지만 겨울이었던지라 강수량이 있었던 날이 적어서 교통량에 큰 변화를 주지 못했습니다. 그로인해 가중치로 사용은 했지만 마찬가지로 큰 효과가 없었습니다.

결론적으로 변수가 워낙 많은 버스 예측 시스템이기에 정확한 정보를 제공할 수 없어 남은 시간을 구하는 공식을 이용해 시간을 구하고 갱신했던 것으로 예상됩니다. 물론 간접적으로 수집한 정보보다 더 정밀한 데이터를 경기도교통청이 가지고 있을것으로 예상되지만 아직 하지 않은 것으로 보아 수지타산이 맞지 않았을 가능성을 생각해 볼 수 있겠습니다.

참조문헌

[국내외 논문]

1. 이진우, 이현미, 이현수. (2017). Levenberg-Marquardt 알고리즘을 이용한 시내버스 지연요소 추정 (Estimation of City Bus Delay Element using Levenberg-Marquardt). 한국전자통신학회 논문지, 12(3), 493-498. <https://doi.org/10.13067/JKIECS.2017.12.3.493>
2. 한국ITS학회. (2009). Markov Chain을 이용한 버스지체시간 예측 (The Bus Delay Time Prediction Using Markov Chain). 한국ITS학회 논문지, 8(3), 1-10. <https://www.kci.go.kr/kciportal/ci/sereArticleSearch/ciSereArtiView.kci?sereArticleSearchBean.artid=ART001362124>

[기술 문서 및 웹 자료]

3. Ramachandran, P., Zoph, B., & Le, Q. V. (2017). Swish: a Self-Gated Activation Function. arXiv preprint arXiv:1710.05941. (Retrieved from Google Brain). <https://arxiv.org/abs/1710.05941>
4. Wikidocs. (n.d.). Swish Activation Function. <https://wikidocs.net/175381>
5. Velog (iissaacc). (n.d.). Swish-function. <https://velog.io/@iissaacc/Swish-function>
6. Tistory (eehoeskrap). (n.d.). Batch Normalization. <https://eehoeskrap.tistory.com/430>
7. Wikidocs. (n.d.). AdamW Optimizer. <https://wikidocs.net/195019>